Chapter 9 — Errors, Exceptions and Files

Programs meet bad input, missing files, full disks and broken networks. Python reports these situations with **exceptions** — objects that interrupt normal flow and travel up the call stack until some code handles them. Most of the time the thing that fails is I/O, so this chapter pairs error handling with working on files: reading and writing text and bytes, paths, CSV, JSON, pickle, temporary files and whole-folder operations.

Scripts are in `code/`; every file operation happens inside a temporary directory that is removed afterwards. Output is real (Python 3.14; code is 3.12-compatible).

What you'll learn

#	Topic	Script
9.1	What exceptions are, tracebacks, the built-in hierarchy	`01_exceptions.py`
9.2	`try` / `except` / `else` / `finally`	`01_exceptions.py`
9.3	Raising, re-raising, custom exception classes	`01_exceptions.py`
9.4	Exception chaining (`raise … from …`), `add_note`	`01_exceptions.py`
9.5	EAFP vs LBYL	`01_exceptions.py`
9.6	`ExceptionGroup` and `except*` (3.11+)	`01_exceptions.py`
9.7	`assert`	`01_exceptions.py`
9.8	Context managers and `with`	`02_context_managers.py`
9.9	File I/O: modes, text vs binary, encodings	`03_files_pathlib.py`
9.10	`pathlib`, `shutil`, `tempfile`	`03_files_pathlib.py`
9.11	`csv`, `json`, `pickle` (and why pickle is unsafe)	`04_csv_json_pickle.py`

9.1 Exceptions and the hierarchy

What it is

An **exception** is an object describing an error, **raised** (thrown) at the point of failure. Raising stops the current code immediately; Python then unwinds the call stack — leaving each function in turn — looking for a matching `except` handler. If none is found, the program stops and prints a **traceback**: the chain of calls that led to the error (read it **bottom-up**: the last line is the exception type and message, the line above is where it happened).

```
Traceback (most recent call last):
  File "app.py", line 8, in <module>
    total = parse("12a")
  File "app.py", line 3, in parse
    return int(text)
ValueError: invalid literal for int() with base 10: '12a'
```

Two kinds of problems are distinct:

- **Syntax errors** are found when Python *compiles* the file (`SyntaxError`, `IndentationError`) — nothing runs.

- **Exceptions** happen while the code *runs* (`ZeroDivisionError`, `KeyError`, …) and can be handled.

Analogy: an exception is a fire alarm. Whoever is nearest and equipped (a matching `except`) deals with it; if nobody does, the building (program) is evacuated.

How it works — the class hierarchy

Every exception is an instance of a class deriving from **`BaseException`**. Handlers match by class *and all its subclasses*, so the tree decides what an `except` catches.

```
== part of the built-in hierarchy ==
  BaseException
      BaseExceptionGroup
      Exception
      GeneratorExit
      KeyboardInterrupt
      SystemExit
  ...under Exception (selection):
  LookupError
      IndexError
      KeyError
  ArithmeticError
      FloatingPointError
      OverflowError
      ZeroDivisionError
  OSError
      FileNotFoundError
      IsADirectoryError
      PermissionError
      TimeoutError
  issubclass(ModuleNotFoundError, ImportError): True
```

(The script walks `__subclasses__()` of the real classes; only a selection is printed.)

Class	Raised when	Example
`SystemExit`, `KeyboardInterrupt`, `GeneratorExit`	Program exit, Ctrl-C, generator closing	Deliberately **not** under `Exception` so `except Exception` doesn't swallow them
`ValueError`	Right type, bad value	`int("abc")`
`TypeError`	Wrong type / wrong call	`"a" + 1`, too many arguments
`KeyError` / `IndexError` (both `LookupError`)	Missing key / index out of range	`{}["x"]`, `[][0]`
`AttributeError`	Missing attribute	`None.upper()`
`NameError` (→ `UnboundLocalError`)	Unknown name	typo in a variable
`ZeroDivisionError` (`ArithmeticError`)	Division by zero	`1 / 0`
`OSError` family	OS-level failures	`FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `TimeoutError`, `ConnectionError`
`ImportError` (→ `ModuleNotFoundError`)	Import failures	Chapter 8
`StopIteration`	Iterator exhausted	used by `for` internally
`RecursionError` (`RuntimeError`)	Too-deep recursion	Chapter 7
`NotImplementedError` (`RuntimeError`)	Abstract method not overridden	
`AssertionError`	`assert` failed	9.7
`UnicodeDecodeError` (→ `ValueError`)	Bytes aren't valid in the codec	9.9

In the interview

30-second answer: "An exception is an object raised on error; it unwinds the stack until an `except` clause whose class matches (including subclasses) handles it, otherwise the program ends with a traceback. All exceptions derive from `BaseException`; application errors derive from `Exception`, while `SystemExit`, `KeyboardInterrupt` and `GeneratorExit` sit directly under `BaseException` so generic handlers don't catch them."

- *Why is `KeyboardInterrupt` not an `Exception`?* → So `except Exception:` doesn't stop Ctrl-C from working.
- *How do you read a traceback?* → Bottom line = type and message; the frames above show the call path, most recent last.

Self-check

1. Common parent of `KeyError` and `IndexError`? → `LookupError`.
2. Is `SyntaxError` catchable in the file that contains it? → No — the file never compiles, so nothing in it runs.
3. What is `FileNotFoundError` a subclass of? → `OSError`.

TL;DR

Exceptions are objects that unwind the stack until matched by class. Know the tree: `BaseException` → `Exception` → your errors.

9.2 `try` / `except` / `else` / `finally`

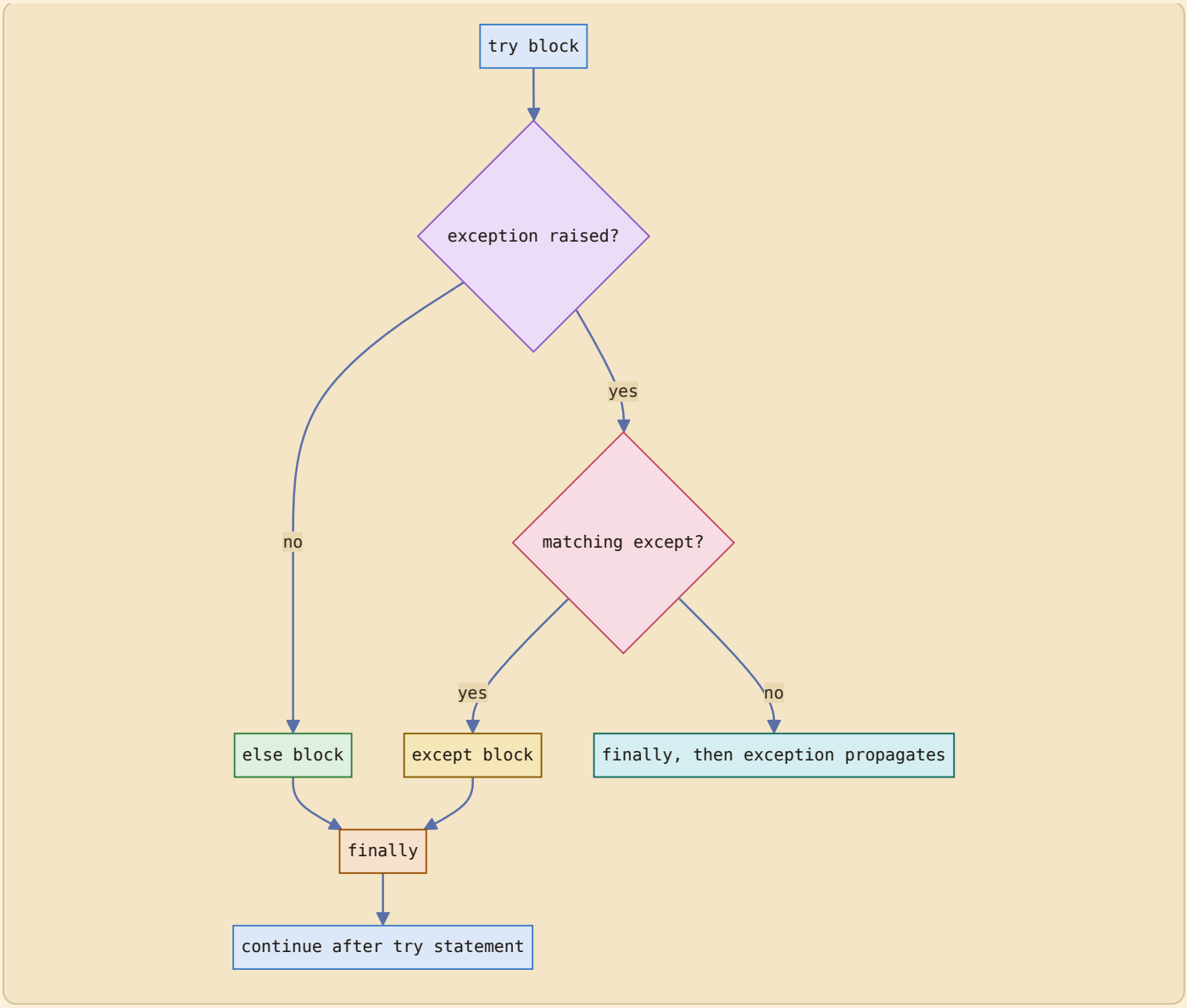
What it is

The statement for handling exceptions:

```
try:
    risky()                 # code that might fail
except SomeError as exc:    # runs if SomeError (or subclass) was raised in try
    handle(exc)
else:                       # runs only if try raised NOTHING
    use_result()
finally:                    # runs ALWAYS: success, handled, unhandled, return, break
    clean_up()
```

How it works

1. Run the `try` block.
2. If an exception occurs, skip the rest of `try` and check `except` clauses **top to bottom**; the **first** matching one runs. `except (A, B) as e:` catches either.
3. If nothing was raised, run `else`.
4. Run `finally` in every case — even if an exception is propagating, or the block `return`s.
5. If no clause matched, the exception continues up the stack (after `finally`).




```python
def parse_age(text):
    try:
        age = int(text)
    except ValueError:
        print(f"  except: {text!r} is not a number")
        return None
    else:
        print(f"  else:   parsed {age}")
        return age
    finally:
        print("  finally: always runs")
```

```
== try / except / else / finally ==
  else:   parsed 42
  finally: always runs
  except: 'forty' is not a number
  finally: always runs
```

`finally` ran even though both branches `return`ed.

Why `else`? It keeps the `try` block tiny — only the line that can raise the error you're handling. Code in `else` isn't protected, so a bug there won't be silently caught by the wrong handler.

Trade-offs / rules of thumb

Do	Don't
Catch the **most specific** exception you can handle	`except:` (bare) — also catches `KeyboardInterrupt` and `SystemExit`
Order handlers specific → general	Put `except Exception` first (it would shadow the rest)
Keep `try` blocks small	Wrap 50 lines in one `try`
Handle where you can *do something* (retry, default, message)	Catch just to `pass` (silently hiding bugs)
Use `finally` / `with` for cleanup	`return` inside `finally` — it swallows exceptions (Python 3.14 emits a `SyntaxWarning`, PEP 765)

When not to use it

Don't use exceptions for ordinary control flow when a simple check or API already exists (`dict.get`, `str.isdigit` for trusted formats). And don't catch errors you can't handle — let them reach a top-level handler that logs them.

Common mistakes

- `except Exception as e: pass` — the bug disappears and resurfaces far away.
- Referencing `e` after the `except` block — Python **deletes** the `as` name at the end of the handler (to break reference cycles), so it's a `NameError` later. Save it to another variable if needed.
- Catching `Exception` around code that includes your own bugs (`TypeError` from a typo) and reporting "invalid input".

Real-world examples

- **Logging instead of hiding**: at the top level, `logging.exception("msg")` (inside an `except`) logs the message *plus* the full traceback; the `traceback` module (`traceback.print_exc()`, `traceback.format_exc()`) gives you the same text as a string or on stderr. See Chapter 14.
- **Warnings are not exceptions**: `warnings.warn("old_api() is deprecated", DeprecationWarning)` prints a message but lets the program continue. Libraries use this for deprecations; `python -W error` (or pytest's `-W error`) turns warnings into exceptions so you catch them in CI.
- Web frameworks (Django, Flask, FastAPI) wrap every request in a top-level handler that turns uncaught exceptions into a 500 response and a log entry, so one bad request doesn't crash the server.
- `requests`: `resp.raise_for_status()` raises `requests.HTTPError` for 4xx/5xx; all its errors derive from `requests.RequestException`, so callers can catch one base class.

In the interview

> **30-second answer:** "`try` runs risky code; the first matching `except` handles; `else` runs only if nothing was raised, which keeps the `try` minimal; `finally` always runs for cleanup. Catch specific exceptions, order specific to general, never a bare `except`."

- *When does `else` run?* → Only when `try` completed without an exception.
- *Does `finally` run if `try` returns?* → Yes, just before the function actually returns.
- *Bare `except:` vs `except Exception:`?* → Bare also catches `BaseException` subclasses like `KeyboardInterrupt`/`SystemExit`.

Self-check

1. Two handlers `except Exception` then `except ValueError` — which runs for `int("x")`? → The first (`Exception`); the second is dead code.
2. What's the point of `else`? → Separate "code that may raise the handled error" from "code that runs on success".
3. Is `e` defined after `except ValueError as e:` finishes? → No.

TL;DR

`try` small, `except` specific (first match wins), `else` on success, `finally` always.

9.3 Raising, re-raising and custom exceptions

What it is

- **`raise SomeError("message")`** signals an error yourself — typically when inputs violate your function's contract.
- **Re-raising**: inside an `except`, a bare **`raise`** re-throws the current exception with its original traceback, after you've logged or cleaned up.
- **Custom exceptions** are classes you define (subclassing `Exception`) so callers can catch *your* errors precisely.

How it works

1. Pick the right built-in when one fits: bad value → `ValueError`, wrong type → `TypeError`, missing key-like thing → `KeyError`/`LookupError`, unsupported operation → `NotImplementedError`.
2. For domain errors, define a **base class** for your library/module and specific subclasses. Callers can catch all your errors (`except PaymentError`) or one (`except CardDeclined`).
3. Name them with an `Error` suffix (PEP 8). Add attributes for data the handler needs.

```
class PaymentError(Exception):
    """Base class for this module's errors."""

class CardDeclined(PaymentError):
    def __init__(self, card_last4, reason):
        super().__init__(f"card ending {card_last4} declined: {reason}")
        self.card_last4 = card_last4
        self.reason = reason

class InsufficientFunds(PaymentError):
    pass

def charge(balance, amount):
    if amount <= 0:
        raise ValueError(f"amount must be positive, got {amount}")
    if amount > balance:
        raise InsufficientFunds(f"need {amount}, have {balance}")
    return balance - amount

def reraise_after_logging(x):
    try:
        return 1 / x
    except ZeroDivisionError:
        print("  logging the error, then re-raising")
        raise                           # bare raise keeps the original traceback
```

```
== raising and catching custom exceptions ==
  charge(100, 30) -> 70
  InsufficientFunds: need 500, have 100
  ValueError: amount must be positive, got -5
  caught CardDeclined, attrs: last4=4242 reason=expired

== re-raising ==
  logging the error, then re-raising
  caller still sees: ZeroDivisionError('division by zero')
```

Pass the message to `super().__init__` so `str(exc)` and tracebacks show it.

Trade-offs

Too few custom exceptions → callers parse message strings to distinguish errors (fragile). Too many → a zoo nobody remembers. A module base class plus a handful of meaningful subclasses is the usual sweet spot. Many libraries also inherit from a matching built-in too (`class InvalidAmount(PaymentError, ValueError)`) so generic `except ValueError` code keeps working.

Common mistakes

- Subclassing `BaseException` instead of `Exception` — escapes `except Exception` handlers.
- `raise exc` instead of bare `raise` inside the handler — works, but adds the current line to the traceback; bare `raise` is the idiom.
- Raising strings (`raise "error"`) — a `TypeError` in Python 3; only exception instances/classes can be raised.
- Returning error codes (`return -1`) instead of raising — callers forget to check.

Real-world examples

- `requests.RequestException` → `ConnectionError`, `Timeout`, `HTTPError`…
- Django: `ObjectDoesNotExist` base with per-model `Model.DoesNotExist` subclasses; `ValidationError` for form/model validation.
- `json.JSONDecodeError` subclasses `ValueError`, so old code catching `ValueError` still works.

In the interview

30-second answer: "Raise built-ins like `ValueError` / `TypeError` for contract violations; for domain errors create a base exception subclassing `Exception` plus specific subclasses with useful attributes. Inside a handler, bare `raise` re-throws with the original traceback."

- *Why a base exception class per library?* → Callers can catch everything from your library with one clause.
- *`raise` vs `raise e`?* → Bare `raise` re-raises the active exception unchanged; `raise e` re-raises the object from the current line.

Self-check

1. Which class should custom exceptions subclass? → `Exception` (or a subclass).
2. What does a bare `raise` outside any handler do? → `RuntimeError: No active exception to reraise`.
3. How do you attach data to a custom exception? → Set attributes in `__init__`.

TL;DR

`raise` built-ins for contract errors; custom hierarchy (base + subclasses, `...Error`) for domain errors; bare `raise` to re-throw.

9.4 Exception chaining and notes

What it is

When you catch a low-level error and raise a higher-level one, Python remembers the link:

Syntax	Stored in	Traceback says
`raise New(...)` inside an `except` (implicit)	`new.__context__`	"During handling of the above exception, another exception occurred"
`raise New(...) from orig` (explicit)	`new.__cause__`	"The above exception was the direct cause of the following exception"
`raise New(...) from None`	suppresses the context	Only the new exception is shown

`exc.add_note("…")` (3.11+) attaches extra context lines that appear under the message in the traceback — handy for "which row/file/request was this?".

Why it matters

Translating errors keeps your API clean (callers catch `ConfigError`, not whatever `KeyError`/`ValueError` your internals produced) *without* losing the root cause needed for debugging.

How it works

```
class ConfigError(Exception):
    pass

def load_port(cfg):
    try:
        return int(cfg["port"])
    except (KeyError, ValueError) as exc:
        raise ConfigError("invalid 'port' setting") from exc
```

```
== exception chaining: raise ... from ... ==
  port = 8080
  ConfigError: invalid 'port' setting; __cause__ = KeyError('port')
  ConfigError: invalid 'port' setting; __cause__ = ValueError("invalid literal for int() with base 10:
'eighty'")
  implicit: __context__ = KeyError('missing')
  formatted traceback of an explicit chain (last lines):
    | Traceback (most recent call last):
    | KeyError: 'port'
    |
    | The above exception was the direct cause of the following exception:
    |
    | Traceback (most recent call last):
    | ConfigError: invalid 'port' setting

== add_note (3.11+) ==
   invalid literal for int() with base 10: 'x1' | notes: ['while reading row 7 of prices.csv']
```

(The script prints only the unindented traceback lines, to keep it short.)

Common mistakes

- Wrapping errors *without* `from` and without a message — the new error hides what really happened.

- Using `from None` everywhere — you throw away debugging information. Use it only when the original is genuinely noise.

In the interview

> **30-second answer:** "Raising inside an `except` chains implicitly via `__context__`; `raise X from e` sets `__cause__` and says 'direct cause' in the traceback; `from None` hides the chain. Use explicit chaining when translating low-level errors into your own exception types. `add_note` adds context in 3.11+."

Self-check

1. Attribute set by `raise A from b`? → `A.__cause__ = b`.
2. How do you hide the original exception? → `raise New(...) from None`.

TL;DR

Translate errors with `raise MyError(...) from exc`; add context with `add_note`.

9.5 EAFP vs LBYL

What it is

Two styles for dealing with things that might not be there:

- **LBYL — Look Before You Leap**: check first, then act. `if key in d: v = d[key]`.
- **EAFP — Easier to Ask Forgiveness than Permission**: just try, and handle the exception. `try: v = d[key] except KeyError: ...`.

Both terms are in the official Python glossary, which calls EAFP the common Python style.

Why it matters

EAFP avoids **race conditions** — the time-of-check-to-time-of-use (TOCTOU) gap. `if os.path.exists(p): open(p)` can still fail if another process deletes the file between the check and the open; `try: open(p) except FileNotFoundError:` has no such gap. EAFP also avoids doing work twice (the check *and* the lookup) and plays well with duck typing (just call the method rather than checking the type).

```python
stock = {"apple": 3}
qty = stock["pear"] if "pear" in stock else 0      # LBYL
try:                                               # EAFP
    qty = stock["pear"]
except KeyError:
    qty = 0
stock.get("pear", 0)                               # the idiomatic one-liner
```

```
LBYL: 0
EAFP: 0 | idiomatic: stock.get('pear', 0) = 0
```

Trade-offs

	EAFP	LBYL
Races (files, shared state)	Safe	Check can go stale
Speed when failure is **rare**	Fast (try is almost free when nothing is raised, 3.11+ "zero-cost")	Pays for the check every time
Speed when failure is **common**	Raising/catching is relatively expensive	Cheaper
Readability	Can hide which line failed if `try` is big	Explicit

When not to use it

Prefer LBYL when failure is the common case, when the "try" would have side effects you can't undo halfway, or when a dedicated API exists (`dict.get`, `getattr(obj, "x", default)`, `str.isdigit()` for simple inputs).

In the interview

> **30-second answer:** "LBYL checks before acting; EAFP attempts and catches. Python favours EAFP: it avoids race conditions like checking a file exists then opening it, and it's cheap when errors are rare. Use LBYL when failure is frequent or partial work is hard to undo."

Self-check

1. Why is `if os.path.exists(p): open(p)` fragile? → The file can vanish between check and open.
2. Idiomatic "get with default" for dicts? → `d.get(key, default)`.

TL;DR

Python prefers EAFP (try/except) — race-free and fast when errors are rare; use LBYL when failure is common.

9.6 `ExceptionGroup` and `except*` (Python 3.11+)

What it is

Sometimes **several** independent errors happen at once — concurrent tasks, validating many fields, running many cleanup callbacks. An **`ExceptionGroup`** (PEP 654) wraps a list of exceptions into one raisable object. **`except*`** handles the matching *part* of a group: each `except*` clause receives a sub-group of the matching exceptions, and anything unhandled is re-raised in a new group.

How it works

```
def run_all():
    errors = []
    for task, fn in [("a", lambda: 1 / 0), ("b", lambda: int("z")),
                     ("c", lambda: 42), ("d", lambda: [][1])]:
        try:
            fn()
        except Exception as exc:
            exc.add_note(f"task {task}")
            errors.append(exc)
    if errors:
        raise ExceptionGroup("some tasks failed", errors)

try:
    run_all()
except* ZeroDivisionError as eg:
    print("  math errors: ", [(repr(e), e.__notes__) for e in eg.exceptions])
except* (ValueError, IndexError) as eg:
    print("  input errors:", [repr(e) for e in eg.exceptions])
```

```
== ExceptionGroup and except* (3.11+) ==
  math errors:  [("ZeroDivisionError('division by zero')", ['task a'])]
  input errors: ['ValueError("invalid literal for int() with base 10: \'z\'")', "IndexError('list
index out of range')"]
```

Unlike plain `except`, **more than one** `except*` clause can run for one group.

Rules: `except` and `except*` can't be mixed in the same `try`; `eg` is always a group (use `eg.exceptions`); `ExceptionGroup` holds `Exception` subclasses, `BaseExceptionGroup` can hold any.

Real-world examples

- `asyncio.TaskGroup` (3.11+) raises an `ExceptionGroup` when child tasks fail.
- The Trio async library pioneered this model and was a key motivation for PEP 654.

When not to use it

Ordinary single errors — use regular `except`. Groups are for genuinely concurrent/independent failures.

In the interview

30-second answer: "An `ExceptionGroup` bundles several exceptions; `except*` lets each clause handle the matching subset, and unhandled ones are re-raised. It's used by `asyncio.TaskGroup` when multiple tasks fail."

Self-check

1. Can two `except*` clauses both run for one group? → Yes.
2. Where would you meet these in the stdlib? → `asyncio.TaskGroup`.

TL;DR

Many errors at once → `ExceptionGroup`; split-handle with `except*`.

9.7 `assert`

What it is

`assert condition, "message"` raises `AssertionError("message")` if `condition` is false. It documents and checks **internal assumptions** — things that should be impossible if your code is correct.

```
def mean(xs):
    assert len(xs) > 0, "mean() of empty list"
    return sum(xs) / len(xs)
```

```
mean([2, 4]) = 3.0
AssertionError: mean() of empty list (disabled entirely by python -O)
```

The crucial caveat

Running Python with **`-O`** (optimise) sets `__debug__` to `False` and **removes all `assert` statements**. So never use `assert` for input validation, security checks or anything that must happen in production — raise a real exception (`ValueError`) instead.

Common mistakes

- `assert (x > 0, "msg")` — a non-empty tuple is always truthy, so this never fails (CPython warns: "assertion is always true").
- Validating user input or permissions with `assert`.

Real-world examples

pytest is built on plain `assert` statements — it rewrites them to show detailed failure messages (`assert result == 5` → shows both sides).

In the interview

> **30-second answer:** "`assert` checks internal invariants and raises `AssertionError`; it's stripped under `python -O`, so use it for debugging and tests, never for validating input."

Self-check

1. What does `python -O` do to asserts? → Removes them.
2. What should validate a function argument from a user? → An explicit `raise ValueError(...)`.

TL;DR

`assert` = developer sanity check and test tool; not validation.

9.8 Context managers and `with`

What it is

A **context manager** is an object that sets something up on entering a `with` block and **guarantees** teardown on leaving it — normally, by `return`/`break`, or by exception. Files, locks, database transactions, temporary directories and timers are typical.

```
with open("data.txt", encoding="utf-8") as f:   # file opened
    data = f.read()
# file closed here, even if read() raised
```

How it works — the protocol

`with EXPR as NAME: BODY` does roughly:

1. Evaluate `EXPR` to get the manager `m`.
2. Call `m.__enter__()`; bind its return value to `NAME`.
3. Run `BODY`.
4. Call `m.__exit__(exc_type, exc, tb)` — with `None, None, None` if the body succeeded, or the exception details if it failed.
5. If `__exit__` returns a **truthy** value, the exception is **suppressed**; otherwise it propagates.

It's `try/finally` packaged as a reusable object.

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        print("  __enter__: timer started")
        return self                     # bound to the name after 'as'

    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"  __exit__: exc_type={exc_type.__name__ if exc_type else None}")
        return False                    # let exceptions propagate
```

```
== class-based: normal exit ==
  __enter__: timer started
  __exit__: exc_type=None
  elapsed measured: True

== class-based: exception inside the block ==
  __enter__: timer started
  __exit__: exc_type=KeyError
  exception still reached caller: KeyError('boom')
```

`@contextlib.contextmanager` writes one as a generator: code before `yield` = enter, the yielded value = `as` target, code after = exit. Wrap the `yield` in `try` to react to errors:

```
@contextlib.contextmanager
def transaction(log):
    log.append("BEGIN")
    try:
        yield log
    except Exception:
        log.append("ROLLBACK")
        raise
    else:
        log.append("COMMIT")
```

```
  log: ['BEGIN', 'UPDATE accounts ...', 'COMMIT', 'BEGIN', 'UPDATE bad ...', 'ROLLBACK']
```

Other `contextlib` helpers used in the script:

Helper	Does	Output
`suppress(FileNotFoundError)`	Ignore listed exceptions in the block	`missing file ignored`
`closing(obj)`	Call `obj.close()` on exit	
`ExitStack()`	Manage a *dynamic number* of managers; exits them in **reverse** order	`open db, cache, queue` → `close queue, cache, db`
Parenthesised `with (A() as a, B() as b):`	Several managers in one statement (3.10+)	

Trade-offs

Class-based managers are best when the object has other methods too (a connection). `@contextmanager` is shortest for one-off setup/teardown. `ExitStack` when the number of resources is only known at runtime.

Common mistakes

- Opening files without `with` and forgetting `close()` — leaks file handles, and buffered writes may not reach disk.
- In a `@contextmanager` generator, forgetting `try/finally` around `yield` — the cleanup won't run if the body raises.
- Returning `True` from `__exit__` by accident → silently swallows every exception.

Real-world examples

- `threading.Lock()`: `with lock:` acquires and always releases.
- `sqlite3` connection: `with conn:` commits on success and rolls back on exception (it does **not** close the connection).
- PyTorch `with torch.no_grad():`, `unittest.mock.patch(...)`, `decimal.localcontext()`, `tempfile.TemporaryDirectory()`.

Decorators and context managers get a deeper treatment (async managers, reentrancy) in Chapter 12.

In the interview

> **30-second answer:** "A context manager implements `__enter__` and `__exit__`; `with` calls enter, runs the block, and always calls exit with any exception info — exit returning true suppresses it. It's a reusable try/finally for resources like files and locks. `contextlib.contextmanager` builds one from a generator; `ExitStack` handles a dynamic set."

- *What does `__exit__` receive?* → `exc_type, exc_value, traceback`, all `None` on success.
- *How do you suppress an exception in a manager?* → Return `True` from `__exit__` (or use `contextlib.suppress`).

Self-check

1. What is bound to `f` in `with open(p) as f`? → The return value of `__enter__` (the file object itself).
2. Order of exits in an `ExitStack`? → Reverse of entry (LIFO).
3. Does `with sqlite3.connect(...) as conn` close the connection? → No, it commits/rolls back.

TL;DR

`with` = guaranteed cleanup. `__enter__`/`__exit__`, or `@contextmanager` with `try` around `yield`. `suppress`, `closing`, `ExitStack` from `contextlib`.

9.9 File I/O — modes, text vs binary, encodings

What it is

`open(path, mode="r", encoding=None, newline=None)` returns a **file object**. In **text mode** it gives and takes `str`, decoding/encoding bytes with an **encoding** (a rule mapping characters to bytes, such as UTF-8). In **binary mode** (`"b"`) it gives and takes raw `bytes`.

Mode	Meaning	If file exists	If missing
`"r"` (default)	read	read from start	`FileNotFoundError`
`"w"`	write	**truncates** to empty	creates
`"a"`	append	writes go to the end	creates
`"x"`	exclusive create	`FileExistsError`	creates
`"+"` added (`"r+"`, `"w+"`)	read and write		
`"b"` added (`"rb"`, `"wb"`)	binary (bytes)		
`"t"` (default)	text (str)		

How it works

```
notes = base / "notes.txt"
with open(notes, "w", encoding="utf-8") as f:    # 'w' creates/truncates
    f.write("first line\n")
    f.writelines(["second line\n", "café ☕\n"])
with open(notes, "a", encoding="utf-8") as f:    # 'a' appends
    print("appended line", file=f)
with open(notes, encoding="utf-8") as f:         # default mode 'r'
    for i, line in enumerate(f, 1):              # lazy, line by line
        print(f"  {i}: {line.rstrip()}")
```

```
== writing and reading text ==
  1: first line
  2: second line
  3: café ☕
  4: appended line
  read() whole file -> 44 characters

== text vs bytes, encodings ==
  bytes on disk: 47 (é is 2 bytes, ☕ is 3 bytes in UTF-8)
  first 12 bytes: b'first line\ns'
  ascii decode -> UnicodeDecodeError ordinal not in range(128) at byte 26
  latin-1 (wrong codec) mangles: 'cafÃ© â\x98\x95'
  errors='replace': caf�� ���

== binary mode, seek/tell ==
  tell after seek(4): 4 | read(3): [4, 5, 6]

== 'x' mode refuses to overwrite ==
  FileExistsError: File exists
```

Reading methods:

Method	Returns	Memory
`for line in f:`	one line at a time (with `\n`)	Constant — best for big files
`f.read()` / `f.read(n)`	whole file / up to `n` chars (bytes in binary)	Whole file
`f.readline()`	next line	One line
`f.readlines()`	list of all lines	Whole file
`f.write(s)`	number of characters written	—
`f.seek(pos)`, `f.tell()`	move / report position	—

Key facts:

1. **Always pass** `encoding="utf-8"` in text mode. Without it Python uses the platform's locale encoding, which differs between machines (historically cp1252 on many Windows systems) — the file you wrote on macOS may not read on Windows. PEP 686 plans to make UTF-8 the default in Python 3.15; until then, be explicit (or run with `-X utf8`).
2. The wrong codec either fails (`UnicodeDecodeError`) or, worse, silently produces **mojibake** (`cafÃ©`). `errors="replace"` / `"ignore"` trade errors for data loss — last resorts.
3. Text mode translates newlines: on reading, `\r\n` and `\r` become `\n` ("universal newlines"); on writing, `\n` becomes the platform's separator. Binary mode never changes bytes.
4. Writes are **buffered**; data is guaranteed on disk after `close()` (or `flush()` + `os.fsync()` for durability).
5. Images, zip files, pickles and anything non-text → binary mode.
6. **File-like objects**: anything with `read()`/`write()` can stand in for a file. `io.StringIO("text")` is an in-memory text file and `io.BytesIO(b"...")` an in-memory binary file — handy in tests and for libraries that expect a file (`csv.reader(io.StringIO(data))`, `pd.read_csv(io.StringIO(...))`).

Common mistakes

- Opening with `"w"` when you meant `"a"` — the file is wiped instantly.
- Omitting `encoding=`.
- `f.read()` on a multi-GB log file — iterate lines instead.
- Building paths with string `+` (`folder + "/" + name`) — use `pathlib`.

Real-world examples

Log processors stream files line by line; `pandas.read_csv(..., encoding="utf-8")` exposes the same encoding choice; almost every "garbled characters" bug report in data pipelines is an encoding mismatch.

In the interview

> **30-second answer:** "`open` returns a file object; use it in a `with` block. Text mode works with `str` and needs an encoding — always pass `utf-8` explicitly; binary mode works with `bytes`. `w` truncates, `a` appends, `x` fails if the file exists. Iterate over the file for large inputs."

- *Difference between `"w"` and `"x"`?* → `w` truncates an existing file; `x` raises `FileExistsError`.
- *Why can 'é' be 2 bytes?* → UTF-8 is variable-width: 1 byte for ASCII, 2–4 for other characters.

Self-check

1. Mode for appending bytes? → `"ab"`.
2. What exception for decoding UTF-8 bytes as ASCII? → `UnicodeDecodeError`.
3. Memory-friendly way to process a huge text file? → `for line in f:`.

TL;DR

`with open(p, mode, encoding="utf-8")`. Text = `str` + encoding, binary = `bytes`. `w` truncates, `a` appends, `x` is safe-create. Stream big files line by line.

9.10 `pathlib`, `shutil`, `tempfile`

What it is

- **`pathlib.Path`** — an object-oriented path. Join with `/`, inspect parts, and read/write/list without juggling strings or `os.path` functions. Works across Windows and POSIX.
- **`shutil`** ("shell utilities") — high-level operations on files and whole trees: copy, move, delete trees, make archives.
- **`tempfile`** — safely creates temporary files and directories with unique names, cleaned up automatically.

How it works — `pathlib`

```
p = base / "reports" / "2026" / "q3.final.csv"
p.name, p.stem, p.suffix, p.suffixes
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text("id,total\n1,9.5\n", encoding="utf-8")
(base / "reports").glob("*/*.csv")
base.rglob("*.md")
```

```
== pathlib ==
  name: q3.final.csv | stem: q3.final | suffix: .csv | suffixes: ['.final', '.csv']
  parent relative to base: reports/2026
  with_suffix('.json'): q3.final.json
  exists: True | is_file: True | size: 15 bytes
  glob('*/*.csv'):    ['q2.csv', 'q3.final.csv']
  rglob('*.md'):      ['readme.md']
  iterdir(reports):   ['2026', 'readme.md']
```

Task	pathlib	older os / os.path
Join	`base / "a" / "b.txt"`	`os.path.join(base, "a", "b.txt")`
Current dir / home	`Path.cwd()`, `Path.home()`	`os.getcwd()`, `os.path.expanduser("~")`
Absolute, symlinks resolved	`p.resolve()`	`os.path.realpath(p)`
Parts	`p.name`, `.stem`, `.suffix`, `.parent`	`os.path.basename`, `splitext`, `dirname`
Exists / type	`p.exists()`, `.is_file()`, `.is_dir()`	`os.path.exists` …
Read / write whole file	`p.read_text()`, `p.write_bytes(b)`	`open(...)`
Make dirs	`p.mkdir(parents=True, exist_ok=True)`	`os.makedirs(p, exist_ok=True)`
Delete file / empty dir	`p.unlink(missing_ok=True)`, `p.rmdir()`	`os.remove`, `os.rmdir`
Rename / replace	`p.rename(t)`, `p.replace(t)`	`os.rename`, `os.replace`
List / search	`p.iterdir()`, `p.glob("*.py")`, `p.rglob("*.py")`, `p.walk()` (3.12+)	`os.listdir`, `glob.glob`, `os.walk`

`open()` and nearly every library accept `Path` objects directly.

How it works — `shutil` and `tempfile`

```
== shutil: copy, move, archive, delete trees ==
  copytree -> ['2026', '2026/q2.csv', '2026/q3.final.csv', 'readme.md']
  moved readme -> True
  archive: reports_bak.zip True
  rmtree backup -> exists? False
  disk_usage works: True

== tempfile ==
  temp file .txt contains: scratch
  deleted after close: True

== atomic replace pattern ==
  settings now: {"v": 2}

temp dir removed: True
```

shutil function	Does
`copy(src, dst)` / `copy2`	Copy a file (+ permission bits / + timestamps and other metadata)
`copytree(src, dst)`	Copy a whole folder tree
`move(src, dst)`	Move/rename, across filesystems if needed
`rmtree(path)`	Delete a folder **and everything in it** — permanent, no recycle bin
`make_archive(base, "zip", root_dir)` / `unpack_archive`	Create/extract zip, tar, gztar…
`which("git")`, `disk_usage(path)`	Find an executable on `PATH`; total/used/free bytes

tempfile API	Gives
`TemporaryDirectory()`	A folder deleted (with contents) when the `with` block ends
`NamedTemporaryFile()`	A file with a real name, deleted on close by default (3.12 adds `delete_on_close=False` for reopening it by name, needed on Windows)
`TemporaryFile()`	Anonymous file — may have no visible name at all
`mkstemp()` / `mkdtemp()`	Low-level: create and return path; **you** delete it

Atomic replace pattern: write the new content to a temp file *in the same directory*, then `os.replace(tmp, target)`. The rename is atomic on the same filesystem, so readers see either the old or the new file — never a half-written one. Tools that save config and state files commonly use this.

Trade-offs

`pathlib` is the modern default; `os`/`os.path` remain for low-level calls (`os.replace`, `os.fsync`, file descriptors) and older code. `shutil` operations are convenient but not transactional — a `copytree` interrupted halfway leaves a partial copy.

Common mistakes

- Hard-coding `"/tmp/myfile"` — predictable names are a security risk and not portable; use `tempfile`.
- `shutil.rmtree` on a path built from user input without checking it.
- Using `str` concatenation or backslashes for paths.
- `Path("a").glob("**/*.py")` vs `rglob("*.py")` — equivalent; `rglob` is shorter.

Real-world examples

pytest's `tmp_path` fixture hands each test a fresh `pathlib.Path` temp directory. Build tools use `shutil.copytree`/`make_archive` for packaging; `shutil.which` is how scripts check that `git` or `ffmpeg` is installed.

In the interview

> **30-second answer:** "`pathlib.Path` is the object-oriented way to build and inspect paths and to read, write, list and glob; `shutil` does high-level file and tree operations like copy, move, rmtree and archives; `tempfile` creates unique temp files and directories that clean themselves up. For safe file updates, write a temp file in the same folder and `os.replace` it."

Self-check

1. Create `a/b/c` even if `a` doesn't exist and don't fail if `c` does? → `Path("a/b/c").mkdir(parents=True, exist_ok=True)`.
2. `Path("x.tar.gz").suffix`? → `".gz"` (`.suffixes` → `['.tar', '.gz']`).
3. Why write to a temp file then `os.replace`? → Atomic update; no half-written file.

TL;DR

`Path` for paths, `shutil` for copying/moving/deleting trees, `tempfile` for throwaway files; atomic writes via temp + `os.replace`.

9.11 `csv`, `json`, `pickle`

What it is

Three standard ways to store structured data in files:

Format	Module	Looks like	Holds	Readable by other languages	Safe to load untrusted data
CSV — comma-separated values	`csv`	Text table	Rows of **strings**	Yes (Excel, databases)	Yes
JSON — JavaScript Object Notation	`json`	Text	dict, list, str, int, float, bool, None	Yes (the web's lingua franca)	Yes
pickle	`pickle`	Binary	Almost any Python object	**No** — Python only	**NO** — can run arbitrary code

CSV

Use the `csv` module, not `line.split(",")` — real CSV has quoted fields containing commas, quotes and even newlines.

```
with open(path, "w", newline="", encoding="utf-8") as f:   # newline="" per csv docs
    w = csv.DictWriter(f, fieldnames=["id", "customer", "total", "placed"])
    w.writeheader()
    for o in orders:
        w.writerow(asdict(o))
with open(path, newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
```

```
  raw file:
    id,customer,total,placed
    1,"Ana, Inc.",19.5,2026-09-01
    2,"Bob ""B"" Ltd",5.25,2026-09-02
  first row: {'id': '1', 'customer': 'Ana, Inc.', 'total': '19.5', 'placed': '2026-09-01'}
  total is a str -> convert yourself: 24.75
  naive split(',') breaks on quoted commas: ['1', '"Ana', ' Inc."', '19.5', '2026-09-01']
  tab-separated: [['a', 'b'], ['1', '2']]
```

- The writer quoted `"Ana, Inc."` and doubled the inner quotes in `"Bob ""B"" Ltd"` automatically.
- **Everything reads back as `str`** — convert types yourself.
- Open with **`newline=""`** (the docs require it) so the `csv` module handles line endings itself; otherwise you can get blank lines between rows on Windows or broken quoted newlines.

- `csv.reader` / `csv.writer` work with lists; `DictReader` / `DictWriter` with dicts keyed by the header; `delimiter="\t"` for TSV; `csv.Sniffer` guesses a dialect.
- For analysis-sized data, `pandas.read_csv` is usually the better tool.

JSON

Python → JSON	JSON → Python
`dict` → object	object → `dict`
`list`, `tuple` → array	array → `list`
`str` → string	string → `str`
`int`, `float` → number	number → `int` or `float`
`True` / `False` → `true` / `false`	→ `True` / `False`
`None` → `null`	→ `None`

`dumps` / `loads` work with strings; `dump` / `load` with files.

```
text = json.dumps(payload, default=to_json_default, indent=2, ensure_ascii=False)
back = json.loads(text)
```

```
  first lines of dumps():
    {
      "orders": [
        {
          "id": 1,
          "customer": "Ana, Inc.",
          "total": 19.5,
          "placed": "2026-09-01"
        },
  loads() types: paid -> bool | discount -> None | placed -> str
  load() from file, customers: ['Ana, Inc.', 'Bob "B" Ltd']
  tuple becomes list: {'pt': [1, 2]}
  int keys become str: {'1': 'a'}
  without default=: Object of type date is not JSON serializable
  JSONDecodeError: Expecting property name enclosed in double quotes at pos 1
```

- Types JSON doesn't know (`date`, `Decimal`, `set`, your classes) need a `default=` function (or pre-conversion, e.g. `dataclasses.asdict`); they come back as plain strings/lists — the round trip is **lossy** for types.
- Tuples become lists; non-string keys become strings.
- JSON requires double quotes; `'single'` quotes (a Python repr) are invalid → `json.JSONDecodeError` (a `ValueError` subclass).
- `indent=2` pretty-prints; `ensure_ascii=False` keeps "é" readable instead of the escape `\u00e9`; `sort_keys=True` gives stable output. `python -m json.tool file.json` pretty-prints from the shell.
- By default `json.dumps(float("nan"))` writes `NaN`, which is **not** valid JSON for strict parsers; pass `allow_nan=False` to get an error instead.

pickle

`pickle` serialises almost any Python object graph (custom classes, dates, nested references) to bytes and back.

```
blob = pickle.dumps(orders)
restored = pickle.loads(blob)
with open(base / "orders.pkl", "wb") as f:                # binary mode!
    pickle.dump(orders, f)
```

```
  round trip equal: True | type: Order | placed: 2026-09-01
  load() from file: 2 orders
```

Security caveat — never unpickle data you don't fully trust. The pickle format lets an object specify, via `__reduce__`, a function to *call* when it's loaded. The script demonstrates with a harmless `print`; an attacker would use `os.system` or similar:

```
class Sneaky:
    def __reduce__(self):
        return (print, ("  !!! code ran during pickle.loads() !!!",))

pickle.loads(pickle.dumps(Sneaky()))
```

```
  loading a crafted pickle...
  !!! code ran during pickle.loads() !!!
  loads() returned: None
```

The code ran merely by *loading*. The official docs carry a prominent warning to this effect.

Other pickle limits: Python-only; tied to your class definitions (rename a class and old pickles break); lambdas, open files, sockets and locks can't be pickled. The current protocol is 5 (`pickle.HIGHEST_PROTOCOL`).

Trade-offs / when to use which

- **Config, APIs, data exchange** → JSON (or TOML for human-edited config, which Python reads with `tomllib`).
- **Tabular data for spreadsheets/other tools** → CSV (or Parquet via pandas for big data).
- **Short-lived caching of Python objects you created yourself** (e.g. between steps of your own pipeline, `multiprocessing` internals) → pickle.
- **Never** pickle for data crossing a trust boundary (uploads, network, downloaded files).

Common mistakes

- Opening pickle files in text mode (`"w"`) → `TypeError`; always `"wb"`/`"rb"`.
- Forgetting `newline=""` with `csv`.
- Expecting `json.loads` to give back `datetime`s.
- Loading downloaded model files with plain `pickle.load`.

Real-world examples

- **PyTorch** checkpoints (`torch.save`) use pickle; since PyTorch 2.6, `torch.load` defaults to `weights_only=True` to block arbitrary code during loading, and Hugging Face created the **safetensors** format as a pickle-free way to share model weights.
- scikit-learn documents pickle/joblib for model persistence with the same security warning, and points to the `skops` format as a safer option.
- REST APIs (GitHub, Stripe) speak JSON; spreadsheet exports are CSV.

In the interview

> **30-second answer:** "CSV is text rows of strings — use the `csv` module with `newline=''` because of quoting. JSON maps to dict/list/str/number/bool/None, is language-neutral and safe; other types need a `default=` hook and come back as plain values. Pickle stores arbitrary Python objects in binary but is Python-only and unsafe: loading can execute code via `__reduce__`, so never unpickle untrusted data."

- *Why is `pickle.loads` dangerous?* → A pickle can instruct the loader to call arbitrary callables.
- *How do you JSON-encode a `datetime`?* → `default=` function returning `isoformat()`, or a custom `JSONEncoder`.
- *`json.dump` vs `json.dumps`?* → File vs string.

Self-check

1. What type are all values from `csv.DictReader`? → `str`.
2. What does `json.loads(json.dumps((1, 2)))` return? → `[1, 2]`.
3. Mode to write a pickle? → `"wb"`.
4. Safe alternative to pickle for sharing ML weights? → safetensors (or `torch.load(..., weights_only=True)`).

TL;DR

CSV = strings in rows (`csv` module, `newline=""`). JSON = safe, portable, limited types. Pickle = any Python object, binary, **never from untrusted sources**.

Chapter TL;DR

- Exceptions are objects that unwind the stack until an `except` matches by class; `Exception` is the base for application errors, `BaseException` also covers exit and Ctrl-C.
- `try` small, `except` specific (first match wins), `else` on success, `finally` always; never a bare `except:`; the `as e` name is deleted after the handler.
- `raise` built-ins for contract errors, a custom hierarchy for domain errors, bare `raise` to re-throw, `raise X from e` to translate while keeping the cause, `add_note` for context.
- Python prefers **EAFP** (try it) over LBYL (check first) — race-free and fast when failures are rare.
- `ExceptionGroup` + `except*` handle several simultaneous errors (e.g. `asyncio.TaskGroup`).
- `assert` checks invariants and vanishes under `-O` — never for validation.
- `with` + context managers guarantee cleanup (`__enter__`/`__exit__`, `@contextmanager`, `ExitStack`).
- `open(path, mode, encoding="utf-8")` inside `with`; text vs binary; `w` truncates, `a` appends, `x` refuses to overwrite; iterate big files.
- `pathlib` for paths, `shutil` for trees and archives, `tempfile` for throwaway files, temp + `os.replace` for atomic writes.
- CSV (strings, `newline=""`), JSON (safe, portable, limited types), pickle (any object, Python-only, **unsafe on untrusted data**).